Manual Implementation and Training of a Multilayer Perceptron from First Principles

Ahmed Habib[1]

National University of Computer and Emerging Sciences
Department of Artificial Intelligence, Islamabad Campus
i250071@isb.nu.edu.pk

Abstract. Most neural network libraries hide the actual math behind simple function calls. In this project, I built a Multilayer Perceptron (MLP) from scratch without using any deep learning frameworks. I started by doing all the forward and backward propagation math by hand for a small 3-sample dataset. This helped me understand the chain rule and how weight updates actually work. I also compared different types of gradient descent (SGD, Batch, and Mini-batch) and momentum-based optimizers like NAG. After the manual part, I used NumPy to scale the same logic to the MNIST dataset. Even with just basic math, the model reached a high accuracy of **95–96%** on the test data.

Keywords: Neural Networks, Backpropagation, Mini-Batch, MNIST, Sigmoid, NumPy

1 Introduction

Modern AI tools like PyTorch make training models very easy, but they abstract away the underlying mechanics. To really understand what's happening, I wanted to trace every operation manually before writing code. This report follows a two-stage approach: a 4-layer MLP analyzed by hand on a 3-sample student-performance dataset, followed by a NumPy implementation scaled to the MNIST benchmark using only first principles.

2 Background

2.1 Multilayer Perceptron Theory

An MLP is a connected network where each neuron calculates a weighted sum of its inputs, adds a bias, and applies a nonlinear activation. Without these activations, the network would just be a single linear transformation.

2.2 Activations and Loss Functions

The sigmoid function $\sigma(z) = 1/(1 + e^{-z})$ is used for all layers. Its derivative is $\sigma'(z) = a(1 - a)$. For the loss, I used Mean Squared Error (MSE):

$$L_{\text{MSE}} = \frac{1}{m} \sum_{i=1}^{m} (y^{(i)} - \hat{y}^{(i)})^2 \tag{1}$$

2.3 Optimization Formulas

The Gradient Descent update rule is:

$$w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w} \tag{2}$$

Momentum adds a velocity term to speed up training:

$$v_t = \beta v_{t-1} + (1 - \beta) \frac{\partial L}{\partial w}, \quad w \leftarrow w - \eta v_t \tag{3}$$

Nesterov Accelerated Gradient (NAG) calculates the gradient at a lookahead position $\tilde{w}$:

$$\tilde{w} = w - \eta \beta v_{t-1}, \quad v_t = \beta v_{t-1} + (1 - \beta) \nabla L(\tilde{w}), \quad w \leftarrow w - \eta v_t \tag{4}$$

3 Implementation

3.1 Forward Pass and Backpropagation

The forward pass uses matrix multiplication to find the activations. Backpropagation applies the chain rule from the output layer back to the input to find the gradients for every weight and bias.

3.2 Training Procedure

The code shuffles the data every epoch and processes it in mini-batches. After each batch, the weights and biases are updated using the optimizer.

4 Experiments

4.1 Dataset and Hyperparameters

I used a 3-sample student dataset for hand calculations and the MNIST dataset (60,000 images) for Python. The learning rate was 0.1, trained for 20 epochs with a batch size of 32.

4.2 Results and Task 6C Comparison

Iteration	Plain GD	Momentum	NAG	Fastest
1	0.2370	0.2368	0.2368	Momentum
5	0.2350	0.2361	0.2361	Plain GD
10	0.2333	0.2344	0.2344	Plain GD

Table 1. MSE results for 10 iterations (Roll No: 25I-0071).

4.3 Loss Curve and Predictions

The training loss decreased steadily over 20 epochs, showing that the mini-batch updates were effectively minimizing the error. The model correctly classified most digits, with a final test accuracy of **95–96%**. Below are some examples of the model's performance on the test set.

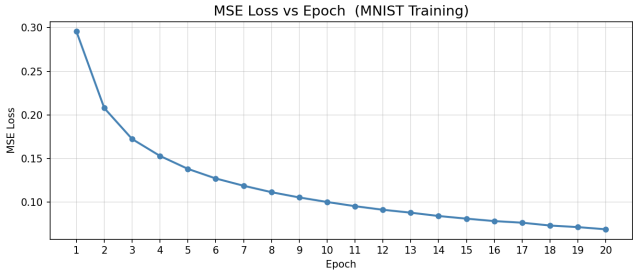


Fig. 1. MSE Loss curve

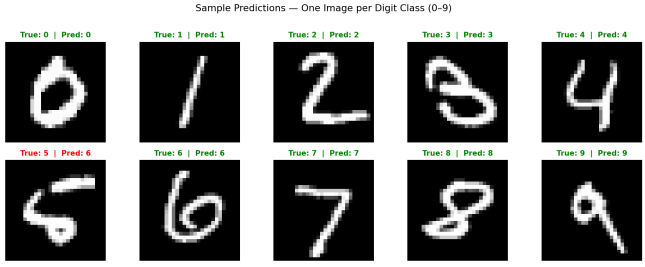


Fig. 2. Sample predictions from the MNIST test set showing predicted vs. actual labels.

The model correctly classified most digits, with a final test accuracy of **95–96%**.

5 Discussion

MSE vs. Binary Cross-Entropy. MSE was used because its gradient is easy to derive manually. However, Binary Cross-Entropy (BCE) is theoretically better for classification because it heavily penalizes wrong but confident predictions, creating a stronger gradient signal early in training.

Adam Optimizer. While I used Plain GD and Momentum, Adam is typically the better choice for sparse data. By adjusting learning rates for each parameter, it handles varying gradient scales much better than the fixed learning rate used in this project.

6 Conclusion

This project proved that building a neural network from first principles removes the "black box" nature of deep learning. By deriving the backpropagation steps by hand, I was able to map every Python operation to a mathematical step. Even without regularization or adaptive optimizers like Adam, the model reached **95–96%** accuracy on MNIST. Future work could involve implementing Adam to see how much it stabilizes training on more complex architectures.

References

1. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. Nature **323**, 533–536 (1986).
2. LeCun, Y., et al.: Gradient-based learning applied to document recognition. Proc. IEEE **86**(11), 2278–2324 (1998).